

Indian Institute of Technology

Department of Cognitive and Brain Sciences

ES 673 - NeuroAI: Neuroscience & Artificial Intelligence
Take Home Assignments

Jaskirat Singh Maskeen

(23110146 | jaskirat.maskeen@iitgn.ac.in)

Instructor: Vaibhav Tripathi

September 11, 2026

Take Home Assignment 2

Exploring Non-Backpropagation Learning Paradigms

Github Repository

2.1 Introduction

As discussed extensively in class, backpropagation has been around for a long time and remains the primary learning paradigm used across various models, be it vision or language models. However, it is computationally very intensive (we will verify it later) and biologically implausible. In this assignment, we implement two such prominent biologically inspired learning paradigms,

2.1.1 Global Backpropagation

In this paradigm, a feedforward network with L layers computes the mapping.

$$\mathbf{y} = f_L(f_{L-1}(\dots f_1(\mathbf{x}; \mathbf{W}_1)\dots; \mathbf{W}_{L-1}); \mathbf{W}_L) \tag{2.1}$$

Given a global loss function $\mathcal{L}(\mathbf{y}, \mathbf{y}^*)$, the gradient with respect to the weights of each layer is obtained by recursively applying the multivariable chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_l} = \boldsymbol{\delta}_l \mathbf{a}_{l-1}^T, \quad \text{where} \quad \boldsymbol{\delta}_l = \left(\mathbf{W}_{l+1}^T \boldsymbol{\delta}_{l+1}\right) \odot \sigma'(\mathbf{z}_l) \tag{2.2}$$

Here, $\boldsymbol{\delta}_l$ denotes the error signal propagated backwards from subsequent layers to layer l, while $\mathbf{a}_{l-1}$ denotes the forward activation of the preceding layer. Hence, computing the gradient at a given layer requires error signals to be recursively propagated from the output through all subsequent layers.

Shortcomings of Backpropagation

1. Computing $\boldsymbol{\delta}_l$ requires propagating errors using the transpose $\mathbf{W}_{l+1}^T$ of the downstream forward weights (Vector Jacobian products). Such weight transport is hard to correlate with the nature of biological synapses.

2. Computing $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_l} = \boldsymbol{\delta}_l \mathbf{a}_{l-1}^T$ requires storing intermediate forward activations until the backward pass. Hence, training memory scales as $\mathcal{O}(L)$ with network depth, leading to high memory requirements for deep networks.

3. Backpropagation couples all layers through a global loss and a sequential forward-and-backwards pass. This contrasts with the mainly local and asynchronous nature of biological neural computation.

2.1.2 Local Learning Algorithms

Local learning paradigms restrict synaptic updates to information available locally at each layer, avoiding the propagation of global error signals through the network (1). A general local learning rule can be expressed as

$$\Delta \mathbf{W}_l = \eta \, \mathcal{F}\left(\mathbf{x}_l, \mathbf{y}_l, \mathbf{s}_l\right),$$

where $\mathbf{x}_l$ and $\mathbf{y}_l$ denote pre- and post-synaptic activity, respectively, and $\mathbf{s}_l$ represents a learning signal or local objective available at layer l. Notice that the weight update does not require gradients propagated from downstream layers, therefore removing the dependence on the global loss:

$$\frac{\partial \mathcal{L}_{\text{global}}}{\partial \mathbf{x}_l} \notin \Delta \mathbf{W}_l.$$

This locality enables layer-wise (and potentially parallel updates) using information available at or near the synapse.
We compare global backpropagation with the following two alternative learning paradigms:

1. **The Forward-Forward Algorithm (2)**: The Forward-Forward (FF) algorithm replaces the forward-backwards pass with two independent forward passes. For each layer l, the network receives *positive* and *negative* examples, denoted by $\mathbf{x}^+$ and $\mathbf{x}^-$, respectively. Positive examples pair the input with the correct label, while negative examples pair it with an incorrect label. Each layer computes a local "goodness" measure, commonly it is the squared activation norm,

$$G_l(\mathbf{x}) = \frac{1}{n_l} \sum_i h_{l,i}(\mathbf{x})^2 \tag{2.3}$$

The layer is trained to produce high goodness for positive examples and low goodness for negative examples, e.g. using

$$\mathcal{L}_l = \log\left(1 + \exp[-(G_l(\mathbf{x}^+) - \theta_l)]\right) + \log\left(1 + \exp[G_l(\mathbf{x}^-) - \theta_l]\right) \tag{2.4}$$

where θ_l is a layer-specific threshold. Thus, each layer can be optimised using only its own forward activations, without receiving error gradients from subsequent layers.

2. **NoProp (3)**: NoProp removes the need for backpropagation between neural blocks by assigning each block an independent denoising objective. Given a target representation $\mathbf{z}_l$, Gaussian noise is added according to a diffusion process,

$$\tilde{\mathbf{z}}_l = \sqrt{\bar{\alpha}_t}\mathbf{z}_l + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}) \tag{2.5}$$

and the corresponding block is trained to recover the clean representation (or the injected noise) from $\tilde{\mathbf{z}}_l$, conditioned on the input. Each block, therefore, minimises its own local denoising loss,

$$\mathcal{L}_l = \mathbb{E}_{t,\epsilon}\left[\left\|\boldsymbol{\epsilon} - \epsilon_{\theta_l}(\tilde{\mathbf{z}}_l, t, \mathbf{x})\right\|^2\right] \tag{2.6}$$

independently of the gradients or activations of downstream blocks.

2.2 Implementation Details

We implement the following paradigms for this assignment:

1. Backpropagation (MLP Baseline)
2. Supervised Forward-Forward (FF)

3. Unsupervised Forward-Forward
4. Discrete-Time NoProp (NoProp)
5. Denoising Diffusion Classifier (Baseline to compare against NoProp)

2.2.1 Network Architectures

The MLP and the FF network have two hidden layers with dimensions 256 and 128, followed by a 10-class linear classification head, hence having about 235k parameters.
NoProp has three independent local denoising blocks with a hidden width of 160, alongside a linear output head. Each block contains an image encoder, a target-state state encoder, and a fusion network, totalling to around 222k parameters.
The Diffusion Baseline has a time-conditioned denoiser with a hidden dimension resulting in roughly 230k parameters.

All the parameters are roughly of same magnitude to better compare the model architectures and their energy requirements.

2.2.2 Forward-Forward

Data Generation

The forward-forward algorithm requires creating negative samples. There are two approaches to do so.

1. Naive Approach: Negative data is generated by adding independent and identically distributed Gaussian or salt-and-pepper noise to images:

   $$\mathbf{x}_{\text{neg}} = \mathbf{x} + \eta, \quad \eta \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$$

   However, this approach fails immediately because it destroys short-range correlations that real images exhibit, such as adjacent pixels sharing stroke width, edge orientation, and smooth gradients.

2. The other approach, as specified in (2) is, that negative data should posses **identical short-range statistics** (smooth strokes, identical textures, but **distorted long-range correlations** (conflicting global shape).

   We implemented this approach as follows:

   (a) A random binary Bernoulli mask $M_0 \in 0, 1^{28\times 28}$ is generated with $p = 0.5$.

   (b) The mask is convolved for four iterations using the separable 2D filter

   $$K = kk^T, \qquad k = [0.25,\ 0.50,\ 0.25],$$

   according to

   $$M_{k+1} = M_k * K.$$

   (c) The smoothed mask is thresholded at 0.5:

   $$M = \mathbb{I}(M_4 \geq 0.5),$$

   producing large, spatially contiguous regions.

   (d) Two real digit images x_1 and x_2 from different samples are combined using the binary mask:

   $$x_{\text{neg}} = M \odot x_1 + (1 - M) \odot x_2.$$

   The positive sample remains the intact image:

   $$x_{\text{pos}} = x_1.$$

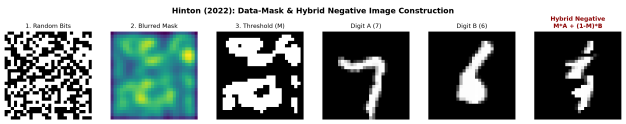


Figure 2.1: Hybrid Mask Generation

3. The above approach works for unsupervised FF; however, for supervised FF, data generation is pretty easy. We use the top-left pixel to one-hot encode the digit class for positive samples. For negative samples, we give them a random incorrect class.

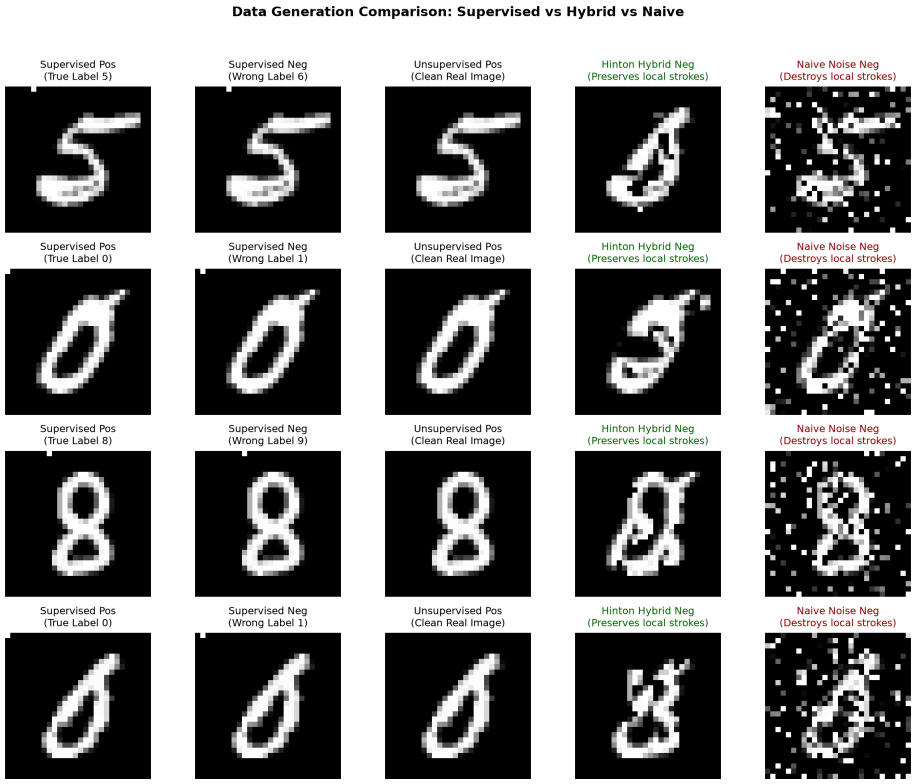


Figure 2.2: Data Masking Pipeline

Goodness Functions

The FF algorithm assigns each layer a scalar *goodness* function $G(\mathbf{y})$ that measures the degree to which the layer's representation $\mathbf{y}$ is correlated with the input. FF trains each layer to produce high goodness for positive examples and low goodness for negative examples (2). A commonly used choice is the squared activation norm,

$$G(\mathbf{y}) = \sum_j y_j^2,$$

which can be interpreted as the total activity, or energy, of the layer's representation. Positive and negative examples are distinguished using a layer-specific threshold θ:

$$p(\text{positive} \mid \mathbf{y}) = \sigma\left(G(\mathbf{y}) - \theta\right).$$

The choice of G determines the direction and structure of the local learning dynamics. For a goodness function to provide a useful local learning signal, it should increase appropriately with feature activation, remain differentiable with well-behaved gradients, and retain sensitivity to the magnitude and selectivity of the representation.

Alternate Goodness Functions

We explored six different goodness functions:

1. Squared Activity:
$$G_{\text{squared}}(\mathbf{y}) = \sum_j y_j^2$$

2. Negative Squared Activity:
$$G_{\text{neg_squared}}(\mathbf{y}) = -\sum_j y_j^2$$

3. Sparse Activity:
$$G_{\text{sparse}}(\mathbf{y}) = \sum_j y_j^2 - 0.20 \sum_j |y_j|$$

4. Margin:
$$G_{\text{margin}}(\mathbf{y}) = \sum_j y_j^2 - 0.40 \sum_j \text{ReLU}(y_j - 1.0)$$

5. Spectral: The hidden activation vector $\mathbf{y} \in \mathbb{R}^d$ is reshaped into an $m \times k$ feature matrix $\mathbf{Y}$, with $\mathbf{Y} \in \mathbb{R}^{16\times 16}$ for $d = 256$. This reshaping introduces a spatial inductive bias (much like a kernel does) by organising the flattened activations into a structured grid, allowing correlations among groups of units to capture spatially coherent patterns in the image.

   A local matrix is then constructed as

$$\mathbf{G} = \frac{1}{m}\mathbf{Y}^T\mathbf{Y} \in \mathbb{R}^{k\times k}.$$

   We consider its dominant eigenvalue, $\lambda_{\max}(\mathbf{G})$, along with the total activation energy ($Tr(\mathbf{G})$). This spectral term measures the extent to which activation variance is concentrated along a dominant direction. The resulting goodness function is defined as

$$G_{\text{spectral}}(\mathbf{y}) = \sum_j y_j^2 + 0.30\, \lambda_{\max}\left(\frac{1}{m}\mathbf{Y}^T\mathbf{Y}\right).$$

6. Hebbian: Inspired by Hebbian plasticity, in which correlated neuronal activity strengthens associations between neurons, we let the goodness metric account for pairwise co-activation. Specifically, the cooperative component is defined as

$$E_{\text{coop}} = \sum_{j\neq k} y_j y_k.$$

   Using the identity

$$\|\mathbf{y}\|_1^2 = \|\mathbf{y}\|_2^2 + \sum_{j\neq k} y_j y_k,$$

   The co-activation term can be computed quickly. We therefore define the Hebbian goodness function as

$$G_{\text{hebbian}}(\mathbf{y}) = \|\mathbf{y}\|_2^2 + \frac{0.50}{d}\left(\|\mathbf{y}\|_1^2 - \|\mathbf{y}\|_2^2\right).$$

   The first term measures individual neuronal activity, while the second rewards coordinated activation across units. Hence, representations characterised by coherent neuron groups, receive higher goodness than those with comparable total activity but weaker inter-unit coordination.

2.2.3 NoProp

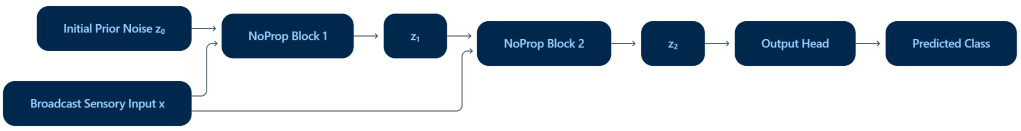


Figure 2.3: Architecture Overview

2.2.4 NoProp Implementation

Target-Space Diffusion and Noise Schedule

Diffusion is applied to the target representation rather than the 784-dimensional input image. For MNIST, each class is represented by a one-hot vector $\mathbf{u}_y \in \Delta^9 \subset \mathbb{R}^{10}$. We use an offset cosine noise schedule (4),

$$f(t) = \cos^2\left(\frac{t/T + s}{1+s}\frac{\pi}{2}\right), \qquad s = 0.008,$$

with

$$\bar{\alpha}_t = \left[1 - \frac{f(t)}{f(0)}\right]_{0.01}^{0.99}.$$

We use $T = 4$ diffusion blocks in the `iso-parameter` configuration and $T = 3$ in the `optimised` configuration. Noisy target states are sampled directly as

$$\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\mathbf{u}_y + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

Dual-Stream NoProp Block

Each block t is an independent neural module. The sensory input $\mathbf{x}$ is broadcast to every block and processed by an image pathway,

$$\mathbf{h}_{\text{img}} = \text{ReLU}\left(\text{LayerNorm}(\mathbf{W}_{\text{img}}\mathbf{x} + \mathbf{b}_{\text{img}})\right).$$

In parallel, the preceding noisy state $\mathbf{z}_{t-1}$ is projected through a separate state pathway,

$$\mathbf{h}_{\text{state}} = \text{ReLU}\left(\text{LayerNorm}(\mathbf{W}_{\text{state}}\mathbf{z}_{t-1} + \mathbf{b}_{\text{state}})\right).$$

The two representations are concatenated and passed through a fusion network to produce class-space logits:

$$\mathbf{z}_{\text{logits}} = \mathbf{W}_2\,\text{ReLU}\left(\mathbf{W}_1[\mathbf{h}_{\text{img}} \| \mathbf{h}_{\text{state}}] + \mathbf{b}_1\right) + \mathbf{b}_2.$$

A softmax projection produces the predicted target representation,

$$\hat{\mathbf{u}}_t = \text{Softmax}(\mathbf{z}_{\text{logits}}) \in \Delta^9.$$

Independent Local Training

Each block is trained independently using the broadcast image, an independently sampled noisy target state, and the clean target representation.

The local objective is an SNR-weighted mean-squared error,

$$\mathcal{L}_t = \frac{w_t}{B}\sum_{i=1}^{B}\|\hat{\mathbf{u}}_t(\mathbf{x}_i, \mathbf{z}_{t-1,i}) - \mathbf{u}_{y,i}\|_2^2,$$

where

$$w_t = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t} - \frac{\bar{\alpha}_{t-1}}{1-\bar{\alpha}_{t-1}}.$$

Each block is optimised using its own AdamW optimiser. This formulation allows the blocks to be trained independently and without storing or propagating network-wide error signals.

Classification Output

A separate linear layer maps the last denoised state to class logits:

$$\hat{\mathbf{y}}_{\text{logits}} = \mathbf{W}_{\text{out}}\mathbf{z}_T + \mathbf{b}_{\text{out}}.$$

The layer is trained independently using standard cross-entropy loss.

Test-Time Reverse Denoising

In inference, the trajectory is initialised from Gaussian noise,

$$\mathbf{z}_0 \sim \mathcal{N}(0, \mathbf{I}),$$

and the blocks are executed sequentially. At each step,

$$\mathbf{z}_t = a_t\hat{\mathbf{u}}_t + b_t\mathbf{z}_{t-1},$$

where a_t and b_t are determined by the diffusion schedule. The final prediction is obtained from the output head applied to $\mathbf{z}_T$.

Hardware Profiling and Energy Tracking

We implemented a hardware profiler using the NVIDIA Management Library (NVML) via `pynvml` to empirically evaluate the computational and memory requirements of the learning paradigms. The experiments were performed on an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB of GDDR6 memory, running Windows 11, Python 3.12, and a CUDA-enabled PyTorch environment.

Power and Energy Measurement

GPU power consumption was sampled through NVML at 50 Hz. The total energy consumption is obtained by integrating the resulting power trace (t_k, P_k) using the trapezoidal rule, and is given by:

$$E_{\text{total}} \approx \sum_{k=1}^{N-1} \frac{P_k + P_{k+1}}{2}(t_{k+1} - t_k).$$

Average power for an experiment is computed using,

$$\bar{P} = \frac{E_{\text{total}}}{t_{\text{end}} - t_{\text{start}}}.$$

Peak Memory Measurement

PyTorch's CUDA memory statistics are used to measure peak algorithmic memory. Before each run, we synchronise the previous GPU operations and record the memory state to get a baseline. Peak allocated memory is given by,

$$M_{\text{peak}} = \frac{\max\left(0,\ M^{\text{peak}}_{\text{allocated}} - M_{\text{baseline}}\right)}{1024^2} \quad \text{[MB]}.$$

2.3 Results

2.3.1 Overview

We benchmark all 5 paradigms on the identical MNIST training (6000 images) and test subsets (1000 images) with a batch size of ($B = 64$) for 15 epochs:

Learning Paradigm	Train Time (s)	Energy (J)	Avg Power (W)	Peak VRAM (MB)	Test Accuracy (%)
Normal MLP	43.79s	232.02 J	5.30 W	13.24 MB	94.70%
Supervised FF (squared)	56.85s	279.38 J	4.91 W	5.87 MB	88.80%
Supervised FF (sparse)	59.88s	191.19 J	3.19 W	5.87 MB	88.30%
Supervised FF (hebbian_wire)	60.69s	243.17 J	4.01 W	5.87 MB	88.10%
Supervised FF (spectral)	74.70s	381.79 J	5.11 W	5.87 MB	87.60%
Supervised FF (margin)	59.35s	220.04 J	3.71 W	5.87 MB	87.50%
Supervised FF (neg_squared)	57.47s	228.53 J	3.98 W	5.87 MB	83.80%
Unsupervised FF (squared)	58.30s	180.59 J	3.10 W	5.67 MB	88.40%
Unsupervised FF (margin)	58.05s	151.26 J	2.61 W	5.67 MB	88.30%
Unsupervised FF (spectral)	74.71s	223.61 J	2.99 W	5.67 MB	88.30%
Unsupervised FF (hebbian_wire)	60.32s	156.00 J	2.59 W	5.67 MB	88.20%
Unsupervised FF (sparse)	60.05s	293.06 J	4.88 W	5.67 MB	87.30%
Unsupervised FF (neg_squared)	58.03s	153.60 J	2.65 W	5.67 MB	51.70%
NoProp	101.15s	373.51 J	3.69 W	3.60 MB	93.80%
Diffusion Classifier	28.65s	81.86 J	2.86 W	7.18 MB	10.30%

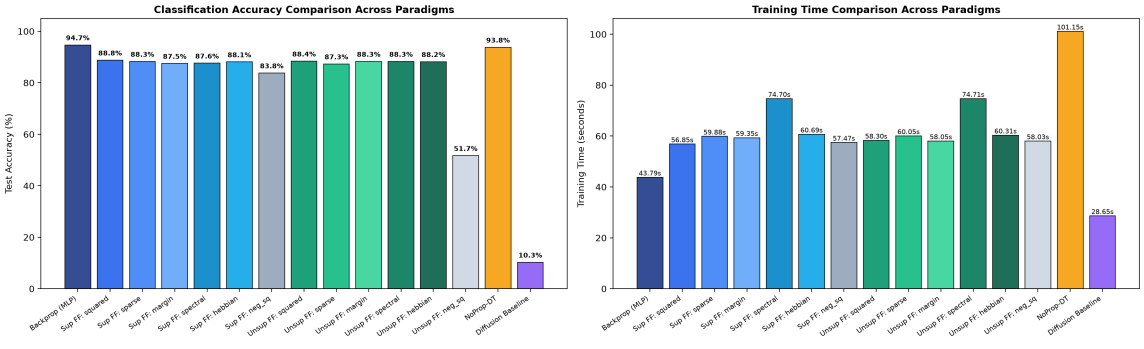


Figure 2.4: Benchmark comparison across all learning paradigms and goodness functions showing classification test accuracy (%) and training time (s).

We observe that across the five learning paradigms, the standard MLP achieves the highest test accuracy at 94.70%, while NoProp attains a comparable 93.80% with the lowest peak VRAM usage of 3.60 MB. Moreover, all variants of Forward-Forward show highly reduced memory consumption, although this comes at the cost of lower accuracy and often longer training times than backpropagation.

Among the FF variants, unsupervised margin achieves the lowest energy consumption at 151.26 J while maintaining 88.30% accuracy. The negative-squared goodness formulation performs particularly poorly in the unsupervised setting, achieving only 51.70% accuracy.

The diffusion classifier consumes the least energy overall (81.86 J), but it only achieves a 10.30% accuracy. This shows that the implemented configuration does not learn a useful classification representation.

2.3.2 Goodness Function Comparison

Across the supervised Forward-Forward variants, the standard `squared` goodness achieved the highest accuracy (88.80%), followed by `sparse` (88.30%) and `hebbian_wire` (88.10%). This indicates that simple activity-based goodness yields a strong, stable local objective, while sparsity and co-activation terms can achieve comparable performance by encouraging selective or coordinated representations. The `spectral` (87.60%) and `margin` (87.50%) variants offered no additional advantage, while `neg_squared` achieved lower accuracy (83.80%).

In the unsupervised setting, where the training set had hybrid negative samples, `squared`, `margin`, and `spectral` achieved the best performance (88.40%, 88.30%, and 88.30%, respectively). This shows that local goodness functions can learn useful digit structure by contrasting intact digits with incoherent digits.

Notice that, `neg_squared` reached only 51.70%.

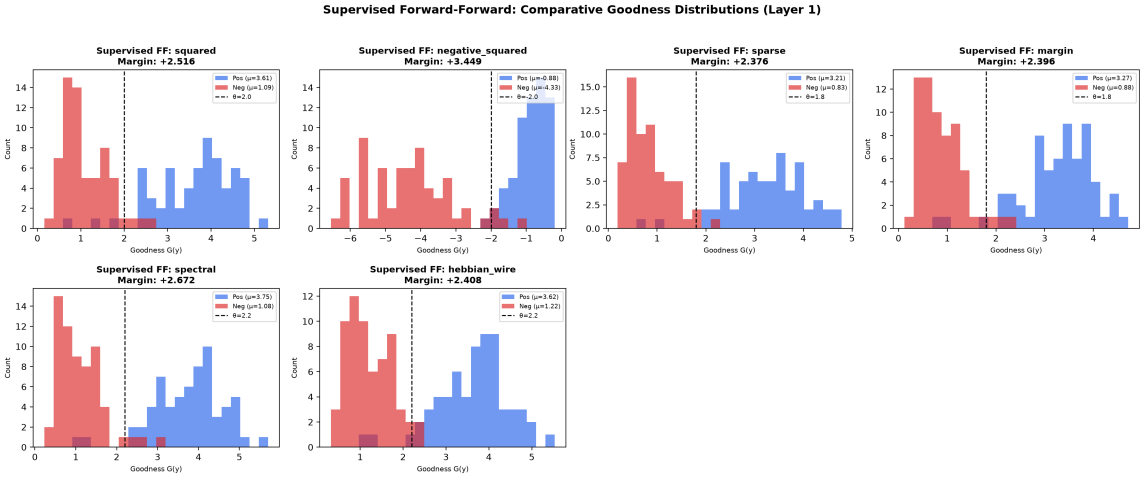


Figure 2.5: Histograms of positive (blue) vs negative (red) goodness distributions across all Supervised FF goodness functions (Layer 1)

2.3.3 NoProp Diffusion Dynamics across Blocks

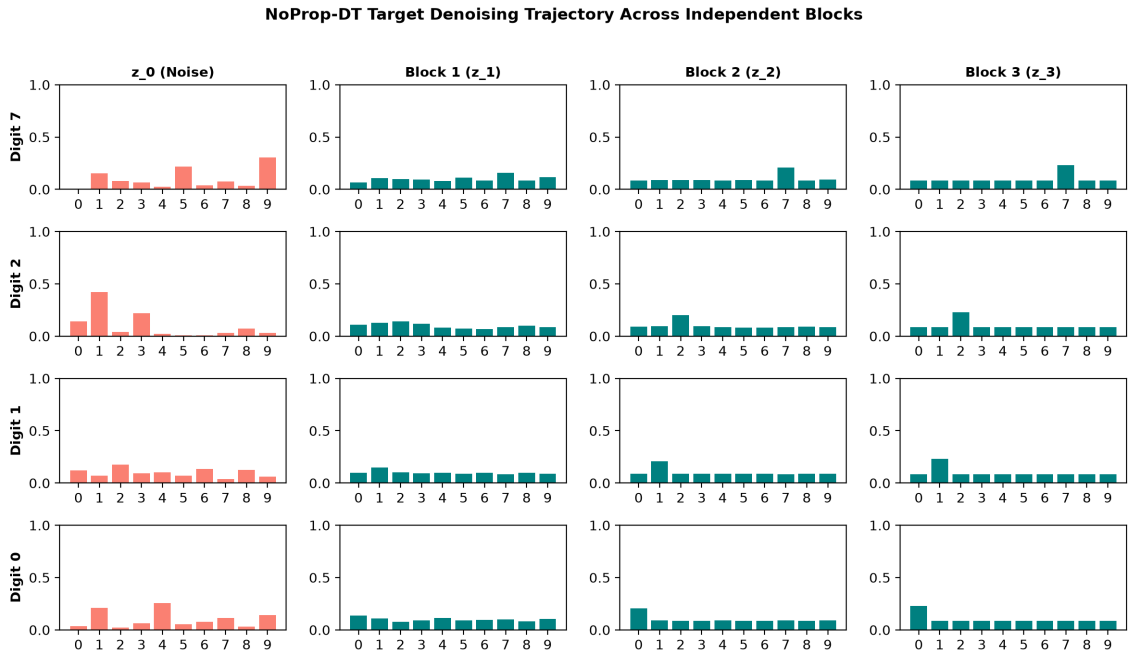


Figure 2.6: Evolution of class state $\mathbf{z}_0 \to \mathbf{z}_1 \to \mathbf{z}_2 \to \mathbf{z}_3$ across independent NoProp blocks. Note that the blocks were trained in complete isolation without any inter-block communication; however, at test time, sequential estimation converges to the true class.

NoProp outperformed the diffusion baseline, because full diffusion models attempt to model the high-dimensional data distribution, whereas NoProp applies diffusion just to the compact target representation space $p(\mathbf{u}_y|\mathbf{x})$.

2.3.4 Representational Differences across Learning Paradigms

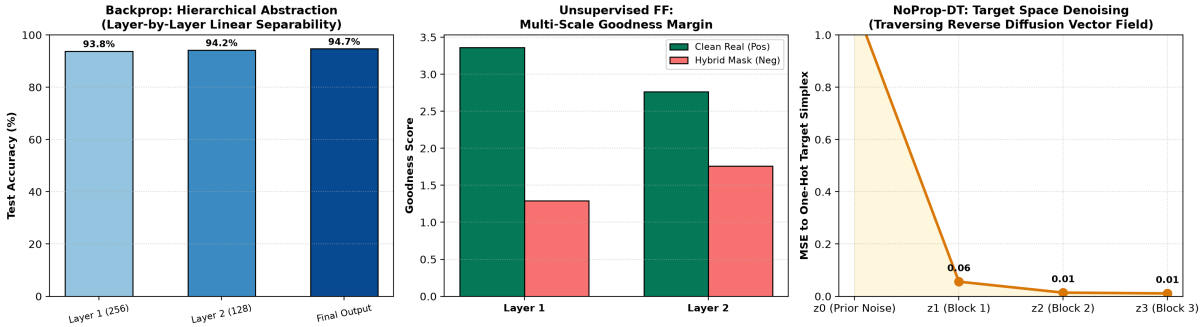


Figure 2.7: (Left) Backpropagation's hierarchical abstraction showing accuracy progressing across layers; (Middle) Unsupervised FF (spectral) multi-scale goodness margin between positive digits and negative digits; (Right) NoProp reverse diffusion trajectory showing MSE contraction to one-hot class across denoising steps $\mathbf{z}_0 \to \mathbf{z}_1 \to \mathbf{z}_2 \to \mathbf{z}_3$.

We analyse the representation of MLP by putting a linear output at each layer and computing the accuracy. Fig. 2.7 shows that backpropagation produces a sort of hierarchical representation, as Layer 1, Layer 2, and the full network achieve 93.8%, 94.2%, and 94.2% accuracy. The high accuracy, just with Layer 1 is due to the size of the first hidden layer (256), that is sufficient enough to represent the MNIST dataset.

Unsupervised FF, however, develops discriminative representations independently at each layer. For the `spectral` goodness function, Layer 1 already separates positive and negative inputs ($\approx$ 3.4 vs. 1.2 mean

goodness), and that is emphasised even more by the next layer. Thus, FF forms a collection of locally discriminative representations rather than a hierarchy.

NoProp represents computation as a temporal denoising trajectory in the 10-dimensional target space. Every block receives the raw image directly and progressively transforms an initial noisy state $\mathbf{z}_0$ toward the target representation, with MSE decreasing from > 1 to 0.06, 0.01, and 0.01 across successive blocks. Representation is therefore encoded in the evolution of the state rather than in spatial feature maps, with later blocks refining the class estimate toward the corresponding class.

2.3.5 Is it really efficient ?

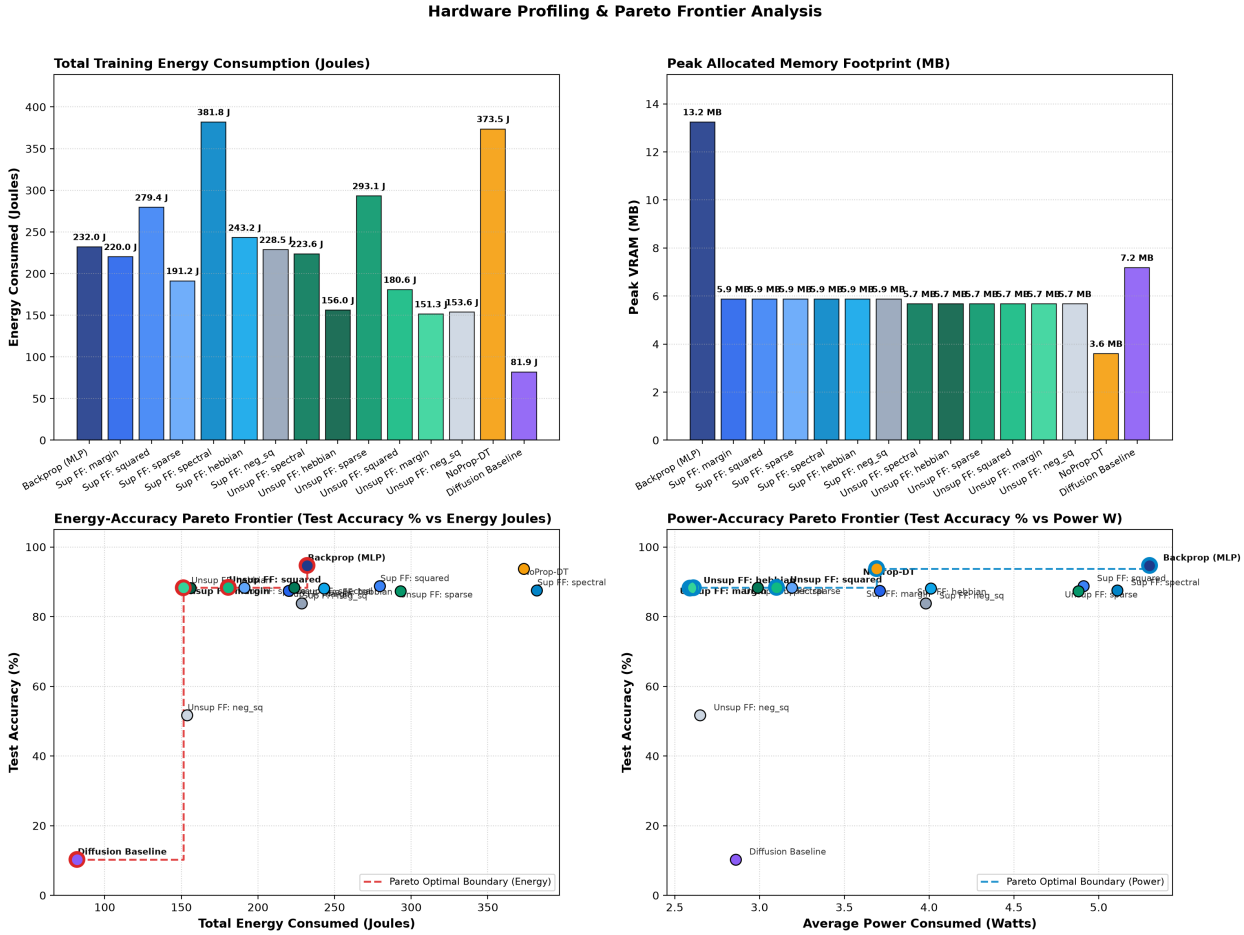


Figure 2.8: (Top-Left) Total energy consumption in Joules ($E = \int P(t)dt$); (Top-Right) Peak allocated VRAM footprint in MB; (Bottom-Left) Energy-Accuracy Pareto frontier; (Bottom-Right) Power-Accuracy Pareto frontier.

Fig. 2.8 show a clear trade-off between accuracy and resource efficiency across paradigms. Backpropagation achieves the highest accuracy (94.7%) but has the largest memory footprint (13.24 MB), whereas NoProp attains comparable accuracy (93.8%) with the lowest peak memory usage (3.60 MB), but at the cost of longer training time and higher total energy consumption.

The energy Pareto frontier indicate that several unsupervised FF configurations are better (points on the graph on top left are better), while the power frontier shows that these methods operate at much lower average power than the MLP.

2.4 Discussion

2.4.1 Biological Plausibility of Forward-Forward and NoProp

Both Forward-Forward (FF) and NoProp exhibit properties relevant to biologically plausible learning. FF eliminates backward error propagation and only uses the forward activity. This allows synaptic updates to depend on locally available pre- and post-synaptic signals. For a squared goodness function, the resulting gradient update has a Hebbian form,

$$\frac{\partial G}{\partial w_{jk}} = 2y_j x_k,$$

which is like local plasticity mechanisms described as Hebbian learning. The positive and negative passes of FF also show a qualitative relation to externally driven and internally generated neural activity (2).

NoProp offers another form of biological plausibility. Each block in the architecture receives the sensory input directly and locally refines a latent state without any inter-block gradient propagation. This iterative state refinement is related to predictive-coding, in which neural systems progressively reduce prediction errors to form good representations (5).

2.4.2 Future of NeuroAI

The results indicate high viability for backpropagation-free learning, with reduced memory and power consumption. However, these methods still have some limitations. FF relies on greedy layer-wise objectives and is sensitive to the construction of correct and useful negative samples (for example, choosing Gaussian noise to create negative samples is not informative!).

NoProp, on the other hand, introduces a sequential denoising trajectory whose errors may accumulate across blocks. This therefore suggests that eliminating backpropagation does not remove the fundamental challenges of representation learning and stability.

However, we can design hardware around the learning algorithm rather than replace backpropagation. Local learning rules are compatible with neuromorphic and in-memory computing substrates, enabling local synapse updates and asynchronous computation. Such hardware, and hence the algorithms on it, are closer to biological neural systems than GPU-based implementations.

References

[1] T. P. Lillicrap, A. Santoro, L. Marris, C. J. Akerman, and G. Hinton, "Backpropagation and the brain," *Nature Reviews Neuroscience*, vol. 21, no. 6, pp. 335–346, Jun 2020. [Online]. Available: https://doi.org/10.1038/s41583-020-0277-3

[2] G. Hinton, "The forward-forward algorithm: Some preliminary investigations," 2022. [Online]. Available: https://arxiv.org/abs/2212.13345

[3] Q. Li, Y. W. Teh, and R. Pascanu, "Noprop: Training neural networks without full back-propagation or full forward-propagation," 2025. [Online]. Available: https://arxiv.org/abs/2503.24322

[4] A. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," 2021. [Online]. Available: https://arxiv.org/abs/2102.09672

[5] R. P. N. Rao and D. H. Ballard, "Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects," *Nature Neuroscience*, vol. 2, no. 1, pp. 79–87, Jan. 1999.